Enzo Sá de Medeiros e Silva

Atividade Prática de implementação Insertion Sort e Merge Sort

Fortaleza, Ceará, Brasil

2026

Introdução

Este documento e seu código-fonte são provenientes da atividade prática de implementação da matéria de Construção e Análise de Algoritmos, lecionada pelo professor Bruno Lopes da Universidade de Fortaleza (Unifor). O documento tem como objetivo implementar os algoritmos Insertion Sort e Merge Sort na linguagem C, coletar tempos de execução para diferentes tamanhos de entrada e cenários, aplicar conceitos de estatística descritiva (média, desvio padrão, intervalo de confiança) na análise dos dados coletados, realizar regressão polinomial sobre os dados coletados e comparar as funções obtidas com as funções teóricas de tempo de execução $T(n)$ e verificar empiricamente se o comportamento observado é consistente com as ordens de crescimento teóricas dos algoritmos, sendo os dois últimos tópicos os objetivos principais deste documento.

No contexto da programação, a ordenação de uma estrutura de dados é frequentemente analisada e discutida no âmbito acadêmico, uma vez que abrange temas como estruturas de dados e análise da eficiência de algoritmos. Dentre os métodos de ordenação criados, o Insertion Sort e o Merge Sort são os mais conhecidos por terem abordagem iterativa simples e a abordagem "dividir para conquistar", respectivamente. Nesse contexto, esse relatório analisa gráficos e tabelas referentes a testes executados na linguagem C usando os 2 algoritmos de ordenação, nos quais foram registrados a quantidade de elementos na estrutura, o tempo necessário para realizar a ordenação e a classificação da organização inicial dos dados em melhor caso, caso médio ou pior caso. No total, foram realizados 50 testes para cada tamanho de entrada n, com $n \in \{1000, 2000, 5000, 10.000, 20.000, 50.000, 100.000, 200.000, 500.000, 1.000.000\}$.

Parte I

Metodologia

1 Metodologia

1.1 Especificações da máquina

Para executar os algoritmos Insertion Sort e Merge Sort na linguagem C e coletar os tempos de execução para diferentes tamanhos de entrada e cenários, foi utilizado um notebook Acer Nitro V 15, equipado com um processador Intel Core i5-13420H de 13ª geração e 24 GB de memória RAM DDR5 (1x8, 1x16) dual channel.

Os algoritmos foram executados na distribuição Ubuntu, por meio do Subsistema do Windows para Linux (WSL), com aproximadamente 11,54 GB de memória RAM disponíveis para uso, em um sistema Windows 11. O desenvolvimento foi realizado utilizando o Visual Studio Code (VS Code).

1.2 Compilador e flags

Para a compilação e a execução do programa `main.c`, responsável pela implementação dos algoritmos de ordenação e pela medição dos tempos de execução, foi utilizado o compilador GCC, versão 15.2.0. Essa escolha justifica-se pelo fato de que os conhecimentos utilizados na implementação dos algoritmos em linguagem C foram adquiridos durante as aulas ministradas pelo professor Bruno Lopes, nas quais foi apresentado o uso desse compilador para transformar o código-fonte em um arquivo executável.

A compilação foi realizada por meio do comando `gcc -o main main.c`, no qual `gcc` representa o compilador utilizado, `-o` especifica o nome do arquivo executável, `main` corresponde ao nome atribuído a esse arquivo e `main.c` representa o arquivo-fonte a ser compilado. Nesse contexto, a única flag utilizada foi `-o`, responsável por definir o nome do arquivo executável gerado.

1.3 Medição de tempo

Para a medição dos tempos de execução dos algoritmos de ordenação Insertion Sort e Merge Sort, utilizou-se a função `clock_gettime()`. No procedimento de medição, essa função recebeu dois argumentos: a constante `CLOCK_MONOTONIC`, que seleciona um relógio adequado à medição de intervalos de tempo, e um ponteiro para uma struct `timespec`, responsável por armazenar o instante registrado em segundos e nanossegundos.

A escolha dessa função justifica-se novamente pela metodologia apresentada pelo professor Bruno Lopes durante as aulas, nas quais foi demonstrada sua utilização para

medir o tempo de execução de algoritmos. Além disso, o enunciado da atividade prática de implementação estabelece explicitamente a obrigatoriedade do uso da função `clock_gettime()`.

1.4 Geração dos vetores para cada cenário

Para a criação dos vetores usados em cada cenário, o enunciado da atividade prática de implementação declara que o melhor caso é representado por um vetor com valores de 1 até n, em ordem crescente. O caso médio é representado por um vetor preenchido com valores gerados aleatoriamente. O pior caso é representado por um vetor com valores de n até 1, em ordem decrescente.

É importante mencionar que o algoritmo Merge Sort possui complexidade $O(n \log n)$ para todos os casos. Portanto, independentemente do vetor a ser ordenado por esse algoritmo, a complexidade será a mesma. Para a padronização do código, a fim de medir o melhor caso do Merge Sort e compará-lo com o Insertion Sort, os vetores foram submetidos às condições propostas pela atividade prática, mesmo que não haja impacto na complexidade do algoritmo Merge Sort.

1.5 Quantidade de repetições realizadas e como os dados foram organizados

A atividade prática exige que cada combinação (algoritmo × cenário × valor de n) seja executada 50 vezes e que, para cada combinação, sejam calculados e apresentados a média do tempo de execução, o desvio padrão, o intervalo de confiança de 95%, o valor mínimo e o valor máximo.

Para o cálculo do intervalo de confiança de 95%, foi utilizada a seguinte fórmula:

$$IC_{95\%} = \bar{x} \pm t_{0{,}975;\,k-1} \frac{s}{\sqrt{k}} \tag{1.1}$$

em que $\bar{x}$ é a média dos tempos para um mesmo tamanho de entrada; s é o desvio padrão amostral; k é a quantidade de medições; e $t_{0{,}975;,k-1}$ é o valor crítico da distribuição t de Student com $k - 1$ graus de liberdade.

Foram avaliados os algoritmos Insertion Sort e Merge Sort nos cenários de melhor caso, caso médio e pior caso. Para cada cenário, foram utilizados dez tamanhos de entrada: 1 000, 2 000, 5 000, 10 000, 20 000, 50 000, 100 000, 200 000, 500 000 e 1 000 000 de elementos. Dessa forma, o experimento compreendeu 60 combinações e totalizou 3 000 medições.

1.5.1 Registro das medições

As medições são armazenadas em arquivos CSV no diretório `data/`. Os resultados do Insertion Sort ficam em `data/insertion/`, enquanto os resultados do Merge Sort ficam em `data/merge/`.

Cada algoritmo possui três arquivos, correspondentes aos cenários avaliados. Os nomes identificam o algoritmo e o cenário, como em `benchmark_insertion_best_case.csv`. As identificações `best_case`, `medium_case` e `worst_case` representam, respectivamente, o melhor caso, o caso médio e o pior caso.

Cada registro de medição contém o tamanho da entrada e o tempo de execução em segundos. Para cada tamanho de entrada, são armazenados 50 registros, resultando em 500 medições por arquivo. Essa estrutura preserva os dados individuais utilizados no cálculo das estatísticas.

1.5.2 Processamento e armazenamento das estatísticas

As tabelas são armazenadas em `data/tables/`, com separação por algoritmo, nos diretórios `insertion/` e `merge/`, e por cenário, nos subdiretórios `best/`, `medium/` e `worst/`.

Cada combinação de algoritmo e cenário possui os arquivos `tabela.csv` e `tabela.png`. O arquivo CSV contém uma linha para cada tamanho de entrada, com a média, o desvio padrão amostral, os limites inferior e superior do intervalo de confiança de 95%, o mínimo e o máximo. O arquivo PNG apresenta os mesmos resultados em formato de imagem. Ao todo, são produzidas seis tabelas, cada uma disponível nos dois formatos.

1.5.3 Organização das representações gráficas

Os gráficos comparativos são localizados nos subdiretórios `best/`, `medium/` e `worst/` de `graphics/`, com o nome `grafico.png`. Cada gráfico apresenta os tempos médios dos dois algoritmos e as barras de erro correspondentes aos intervalos de confiança de 95%.

Os gráficos que comparam os tempos medidos aos modelos teóricos são separados nos diretórios `data/graphics/insertion_theoretical/` e `data/graphics/merge_theoretical/`. Cada diretório contém os arquivos `best_case.png`, `medium_case.png` e `worst_case.png`.

Cada um desses seis gráficos apresenta duas curvas: os tempos médios medidos e o modelo teórico correspondente, utilizando escala linear nos dois eixos. Para o Insertion Sort, são utilizados os modelos $T(n) = cn$, no melhor caso, e $T(n) = cn^2$, nos casos médio e pior. Para o Merge Sort, utiliza-se $T(n) = cn \log_2(n)$ nos três cenários. A constante c é ajustada por mínimos quadrados aos tempos médios de cada combinação de algoritmo e cenário, enquanto a legenda apresenta apenas a expressão geral do modelo.

1.6 Ferramenta para cálculo estatístico, geração de gráficos e cálculo da regressão polinomial

Para a elaboração dos gráficos e das tabelas exigidos pela atividade prática de implementação, utilizou-se a linguagem de programação Python, com o auxílio das bibliotecas de código aberto *pandas*, *SciPy*, *Matplotlib* e *NumPy*. A biblioteca *pandas* organiza as medições, *SciPy* fornece a distribuição t de Student, *Matplotlib* gera as tabelas em imagem e os gráficos e *NumPy* organiza os dados em vetores numéricos, calcula modelos teóricos e gera pontos de curva dos gráficos teóricos.

É importante relatar que a disciplina de Construção e Análise de Algoritmos tem como objetivo desenvolver a capacidade de construir algoritmos e analisar sua eficiência. Nesse contexto, os scripts em Python destinados ao processamento dos dados e à geração das tabelas e dos gráficos foram gerados com o uso de inteligência artificial, por meio do modelo `GPT-6.1 Sol (High)`, da OpenAI. Tais scripts foram, sempre que gerados ou alterados, revisados por um humano, que checou os dados e verificou que estavam corretos e coerentes.

1.7 Cálculo da regressão polinomial

Regressão polinomial é, por definição, técnica que ajusta uma função polinomial aos dados. Para o Insertion Sort, foram utilizados $T(n) = cn$, no melhor caso, e $T(n) = cn^2$, nos casos médio e pior. Nesses casos apenas a constante c é calculada.

Para o Merge Sort, foi utilizado o modelo $T(n) = cn \log_2(n)$, que não constitui uma função polinomial.

A constante c foi estimada pelo método dos mínimos quadrados, minimizando a soma dos quadrados das diferenças entre os tempos médios medidos e os tempos estimados por cada modelo. Esse procedimento permite comparar o crescimento previsto pela análise teórica com o comportamento observado experimentalmente, ajustando a escala dos modelos aos dados obtidos. Caso contrário, a visualização, fora de escala, seria impossível, já que a curva dos dados medidos seria muito pequena em relação ao crescimento da curva teórica

Parte II

Resultados

2 Resultados

2.1 Tabelas

Insertion Sort — Melhor caso

Entrada	Média (s)	Desvio padrão (s)	Intervalo de Confiança 95% (s)		Mínimo (s)	Máximo (s)
			inferior	superior		
1.000	1.4484e-06	3.3463962e-08	1.4388896e-06	1.4579104e-06	1.4e-06	1.55e-06
2.000	2.9244e-06	9.251545e-07	2.661474e-06	3.187326e-06	2.72e-06	9.32e-06
5.000	9.4214e-06	8.4474817e-06	7.0206523e-06	1.1822148e-05	6.72e-06	4.41e-05
10.000	1.34886e-05	5.4958612e-08	1.3472981e-05	1.3504219e-05	1.34e-05	1.369e-05
20.000	3.298e-05	2.6893114e-05	2.5337062e-05	4.0622938e-05	2.692e-05	0.00021624
50.000	7.09006e-05	3.8926495e-06	6.9794321e-05	7.2006879e-05	6.867e-05	8.993e-05
100.000	0.0001415384	8.6298517e-06	0.00013908582	0.00014399098	0.00013745	0.00018095
200.000	0.0002816034	1.1272127e-05	0.0002783999	0.0002848069	0.00027007	0.00032106
500.000	0.000742933	0.0001450499	0.00070171027	0.00078415573	0.00067001	0.00160247
1.000.000	0.00148665	0.00013446057	0.0014484367	0.0015248633	0.00134279	0.00199308

Figura 1 – Tempos de execução do Insertion Sort no melhor caso

Fonte: Elaboração própria.

Insertion Sort — Caso medio

Entrada	Média (s)	Desvio padrão (s)	Intervalo de Confiança 95% (s)		Mínimo (s)	Máximo (s)
			inferior	superior		
1.000	0.0003456094	0.00010434321	0.00031595539	0.00037526341	0.00022957	0.00076277
2.000	0.0010400024	0.00010867125	0.0010091184	0.0010708864	0.00090671	0.00140303
5.000	0.0065976294	0.0014091349	0.0061971577	0.0069981011	0.00588525	0.01357602
10.000	0.024973818	0.0010943069	0.024662819	0.025284817	0.02329345	0.02994448
20.000	0.10175912	0.0035818621	0.10074116	0.10277707	0.09707067	0.11563887
50.000	0.61053041	0.0067769412	0.60860443	0.6124564	0.59964068	0.6366798
100.000	2.4645852	0.018050387	2.4594553	2.469715	2.42533	2.5171242
200.000	9.801811	0.068496858	9.7823445	9.8212776	9.6659105	9.9383386
500.000	61.385399	0.15816437	61.34045	61.430349	60.927554	61.663623
1.000.000	245.33948	0.46994956	245.20593	245.47304	244.51892	247.32343

Figura 2 – Tempos de execução do Insertion Sort no caso médio

Fonte: Elaboração própria.

2.2 Gráficos

Insertion Sort — Pior caso

Entrada	Média (s)	Desvio padrão (s)	Intervalo de Confiança 95% (s)		Mínimo (s)	Máximo (s)
			inferior	superior		
1.000	0.0006164766	0.00012556235	0.00058079218	0.00065216102	0.00045472	0.00097352
2.000	0.0020650934	0.00023513511	0.0019982687	0.0021319181	0.00183424	0.00321233
5.000	0.013191056	0.0018825186	0.01265605	0.013726061	0.01174353	0.02137561
10.000	0.049935811	0.0021891604	0.049313659	0.050557964	0.0473748	0.0607632
20.000	0.19728789	0.004045155	0.19613827	0.19843751	0.19302812	0.21718972
50.000	1.2298799	0.014364194	1.2257976	1.2339621	1.2042308	1.277403
100.000	4.9087708	0.036550558	4.8983832	4.9191584	4.853877	4.9797306
200.000	19.675592	0.098495087	19.6476	19.703584	19.437379	19.899215
500.000	122.26851	0.41527901	122.15049	122.38653	121.48439	123.33869
1.000.000	493.38682	8.5535159	490.95594	495.8177	488.54114	540.63785

Figura 3 – Tempos de execução do Insertion Sort no pior caso

Fonte: Elaboração própria.

Merge Sort — Melhor caso

Entrada	Média (s)	Desvio padrão (s)	Intervalo de Confiança 95% (s)		Mínimo (s)	Máximo (s)
			inferior	superior		
1.000	3.66518e-05	4.8068054e-06	3.5285721e-05	3.8017879e-05	3.459e-05	6.332e-05
2.000	8.03008e-05	1.100578e-05	7.7172992e-05	8.3428608e-05	7.35e-05	0.00013678
5.000	0.0002183206	2.5252591e-05	0.00021114389	0.00022549731	0.0001952	0.0003192
10.000	0.000463352	0.00011554694	0.00043051392	0.00049619008	0.00039999	0.00115037
20.000	0.0009949008	0.00026021644	0.00092094811	0.0010688535	0.0008106	0.00196183
50.000	0.0025172994	0.00032659337	0.0024244826	0.0026101162	0.00212805	0.00368331
100.000	0.0052770364	0.00063354712	0.0050969843	0.0054570885	0.00463728	0.00791115
200.000	0.010669379	0.00046541334	0.01053711	0.010801648	0.00995458	0.01209083
500.000	0.027949433	0.0011059654	0.027635121	0.028263745	0.0264753	0.0323174
1.000.000	0.058480869	0.0019641613	0.05792266	0.059039077	0.05557312	0.06444564

Figura 4 – Tempos de execução do Merge Sort no melhor caso

Fonte: Elaboração própria.

Merge Sort — Caso medio

Entrada	Média (s)	Desvio padrão (s)	Intervalo de Confiança 95% (s)		Mínimo (s)	Máximo (s)
			inferior	superior		
1.000	7.21438e-05	2.6950286e-06	7.1377881e-05	7.2909719e-05	6.904e-05	8.205e-05
2.000	0.0001550996	8.3449919e-06	0.00015272798	0.00015747122	0.00014896	0.00020888
5.000	0.0004284516	2.0450534e-05	0.00042263962	0.00043426358	0.00041229	0.00055537
10.000	0.0009367634	7.8747561e-05	0.00091438359	0.00095914321	0.00087147	0.0011628
20.000	0.0020040002	0.00022961277	0.001938745	0.0020692554	0.00183369	0.00278023
50.000	0.005360046	0.00035814753	0.0052582616	0.0054618304	0.00498115	0.00670166
100.000	0.011256786	0.00067830589	0.011064013	0.011449558	0.01058399	0.01355674
200.000	0.023415058	0.00086373737	0.023169586	0.023660529	0.02224362	0.02674107
500.000	0.062704157	0.0026741659	0.061944168	0.063464147	0.06011247	0.07658385
1.000.000	0.13142976	0.0025253282	0.13071207	0.13214745	0.12724213	0.13791963

Figura 5 – Tempos de execução do Merge Sort no caso médio

Fonte: Elaboração própria.

Merge Sort — Pior caso

Entrada	Média (s)	Desvio padrão (s)	Intervalo de Confiança 95% (s)		Mínimo (s)	Máximo (s)
			inferior	superior		
1.000	3.6194e-05	4.9480567e-07	3.6053378e-05	3.6334622e-05	3.508e-05	3.766e-05
2.000	8.4096e-05	1.7768303e-05	7.9046304e-05	8.9145696e-05	7.44e-05	0.00017034
5.000	0.0002133122	1.8015226e-05	0.00020819233	0.00021843207	0.00019676	0.00026437
10.000	0.000444837	3.6115413e-05	0.00043457311	0.00045510089	0.00041104	0.00060789
20.000	0.000953128	0.00017170441	0.00090433015	0.0010019259	0.00082669	0.00196358
50.000	0.002578134	0.00039196265	0.0024667394	0.0026895286	0.00221414	0.00415325
100.000	0.0051302468	0.00042316031	0.005009986	0.0052505076	0.00465664	0.00682173
200.000	0.010701556	0.00056544858	0.010540858	0.010862255	0.00991943	0.01280549
500.000	0.028664183	0.00098163649	0.028385205	0.028943161	0.02704015	0.03133654
1.000.000	0.059460674	0.0026692499	0.058702082	0.060219266	0.05616574	0.07024884

Figura 6 – Tempos de execução do Merge Sort no pior caso

Fonte: Elaboração própria.

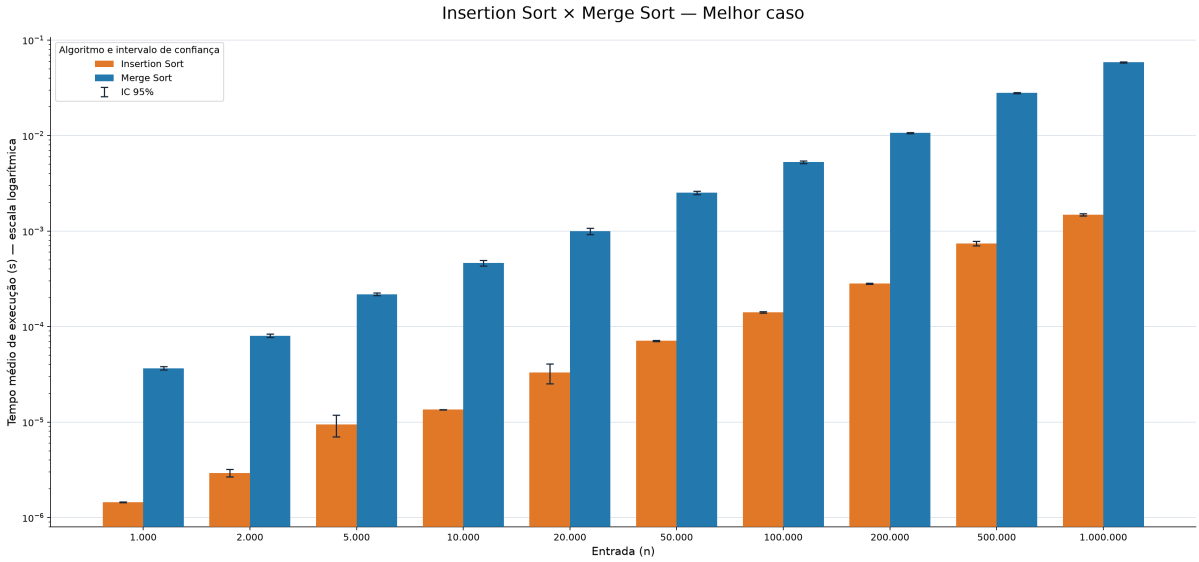


Figura 7 – Tempos de execução do Insertion Sort e do Merge Sort no melhor caso

Fonte: Elaboração própria.

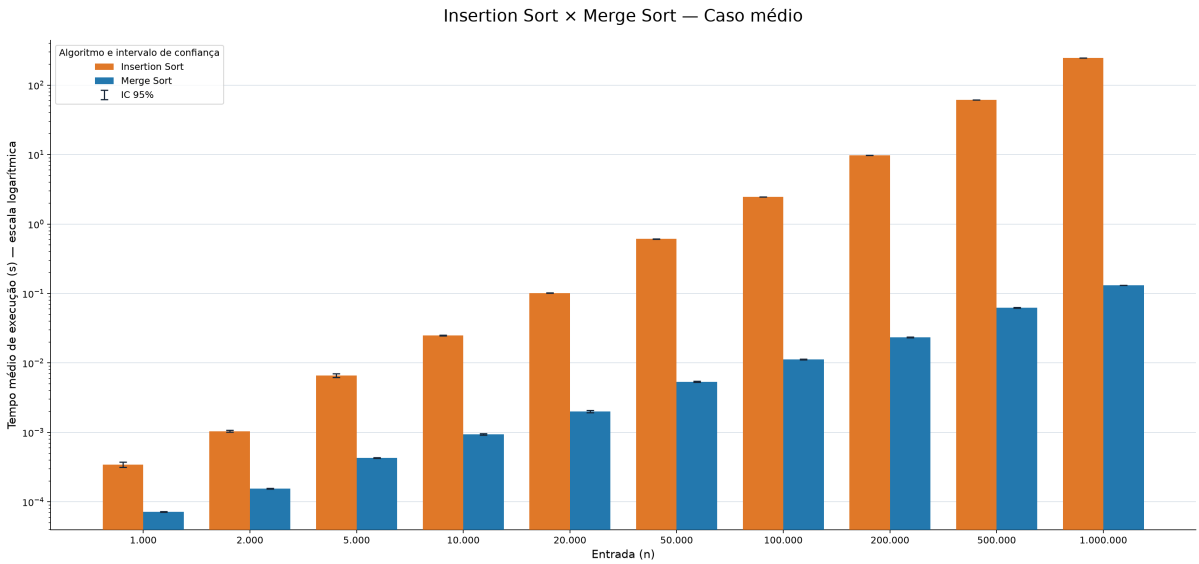


Figura 8 – Tempos de execução do Insertion Sort e do Merge Sort no caso médio

Fonte: Elaboração própria.

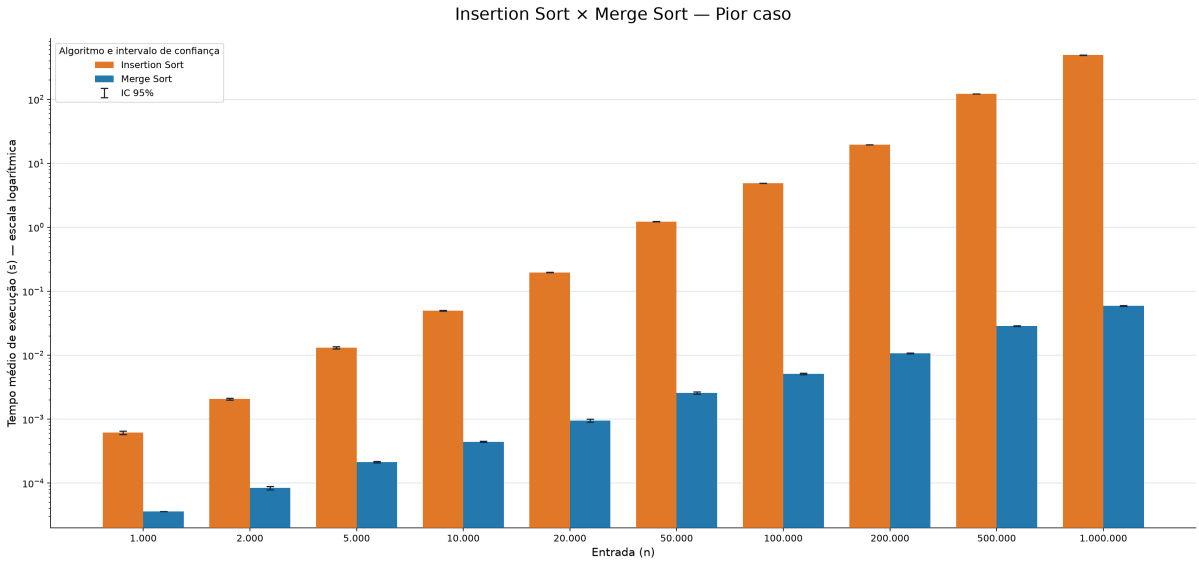


Figura 9 – Tempos de execução do Insertion Sort e do Merge Sort no pior caso

Fonte: Elaboração própria.

2.2.1 Gráficos teóricos

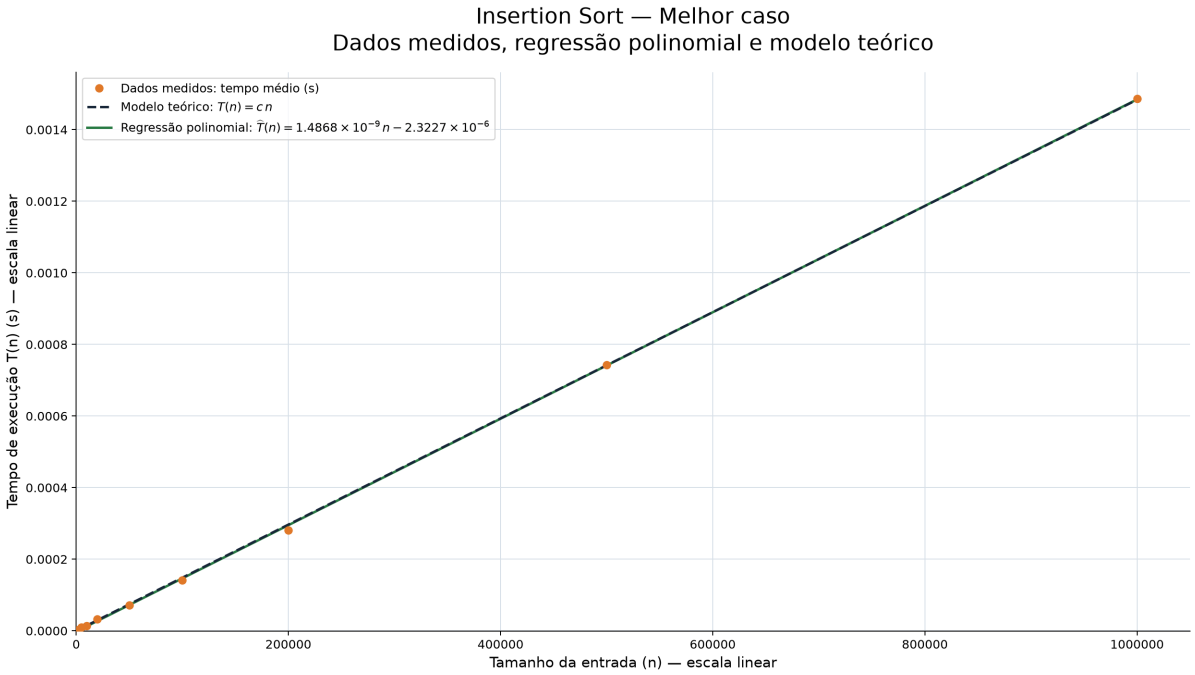


Figura 10 – Tempos de execução do Insertion Sort no melhor caso

Fonte: Elaboração própria.

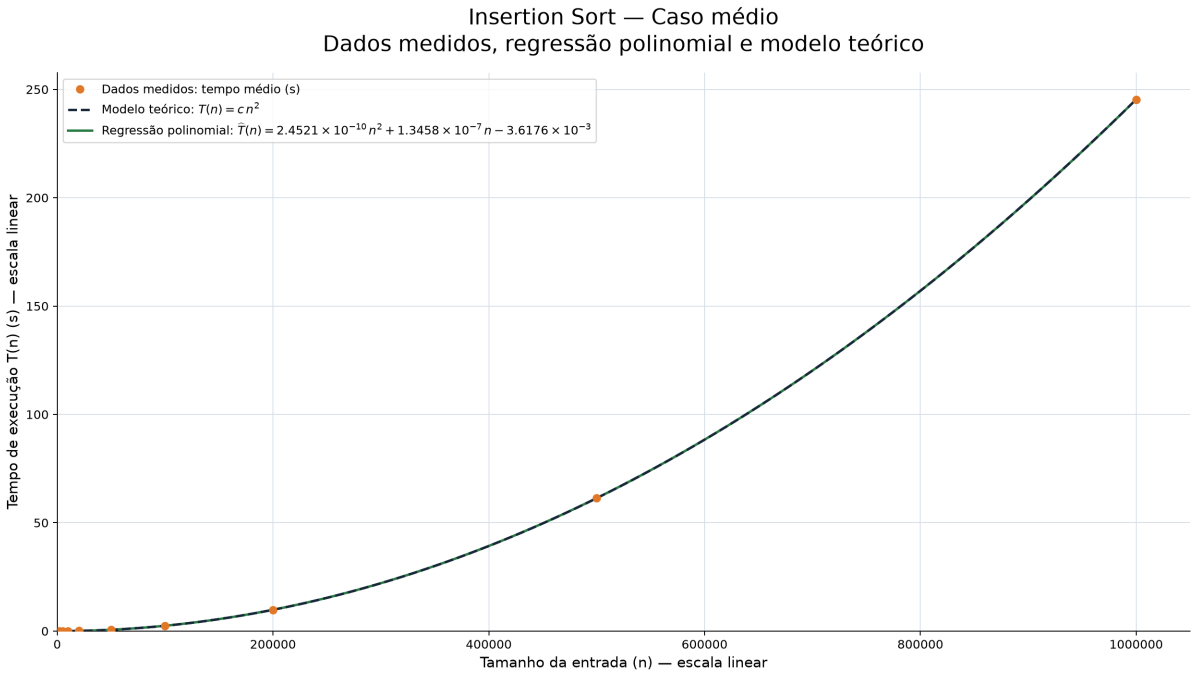


Figura 11 – Tempos de execução do Insertion Sort no caso médio

Fonte: Elaboração própria.

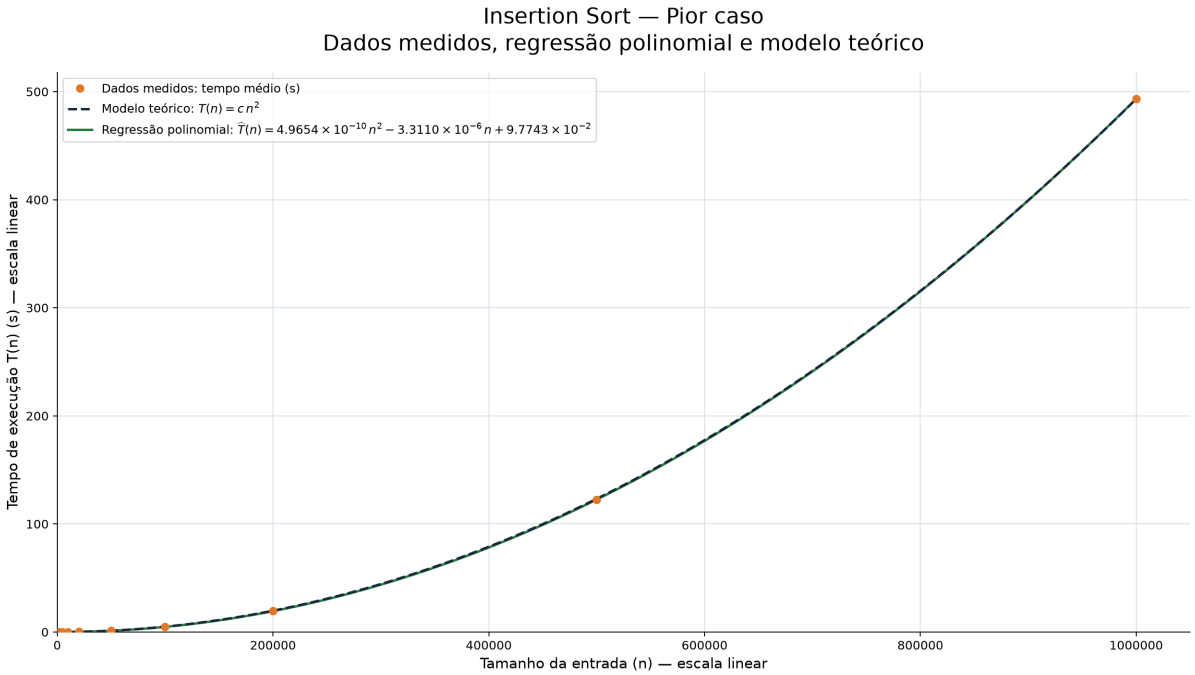


Figura 12 – Tempos de execução do Insertion Sort no pior caso

Fonte: Elaboração própria.

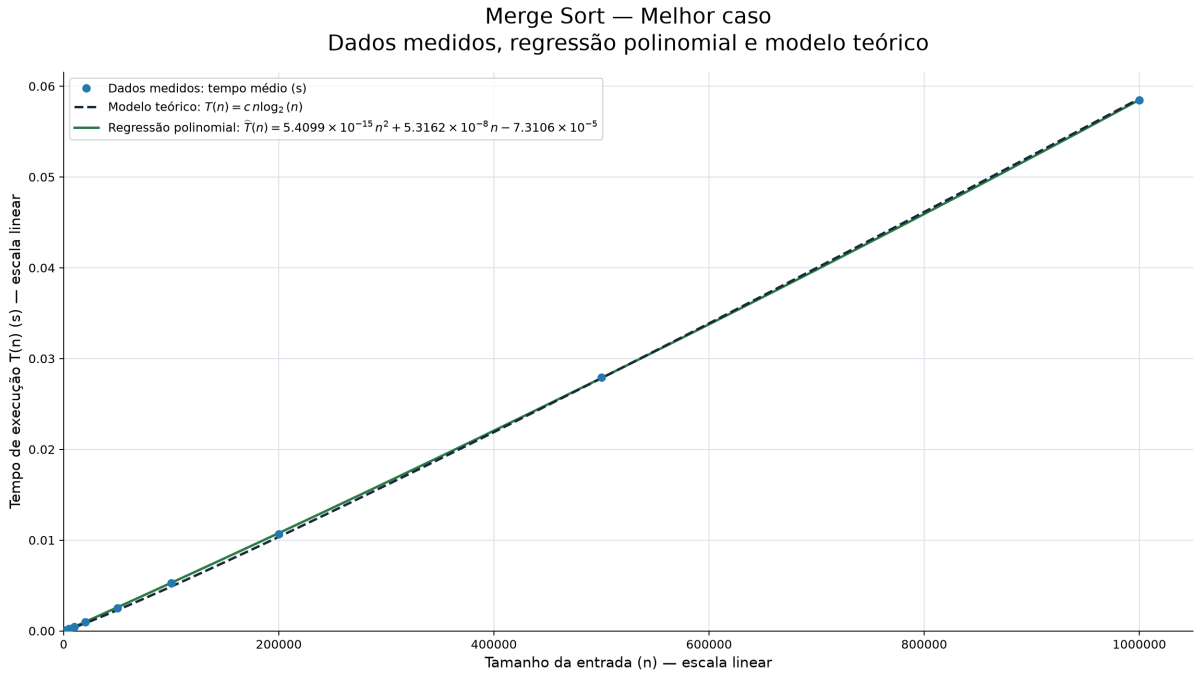


Figura 13 – Tempos de execução do Merge Sort no melhor caso

Fonte: Elaboração própria.

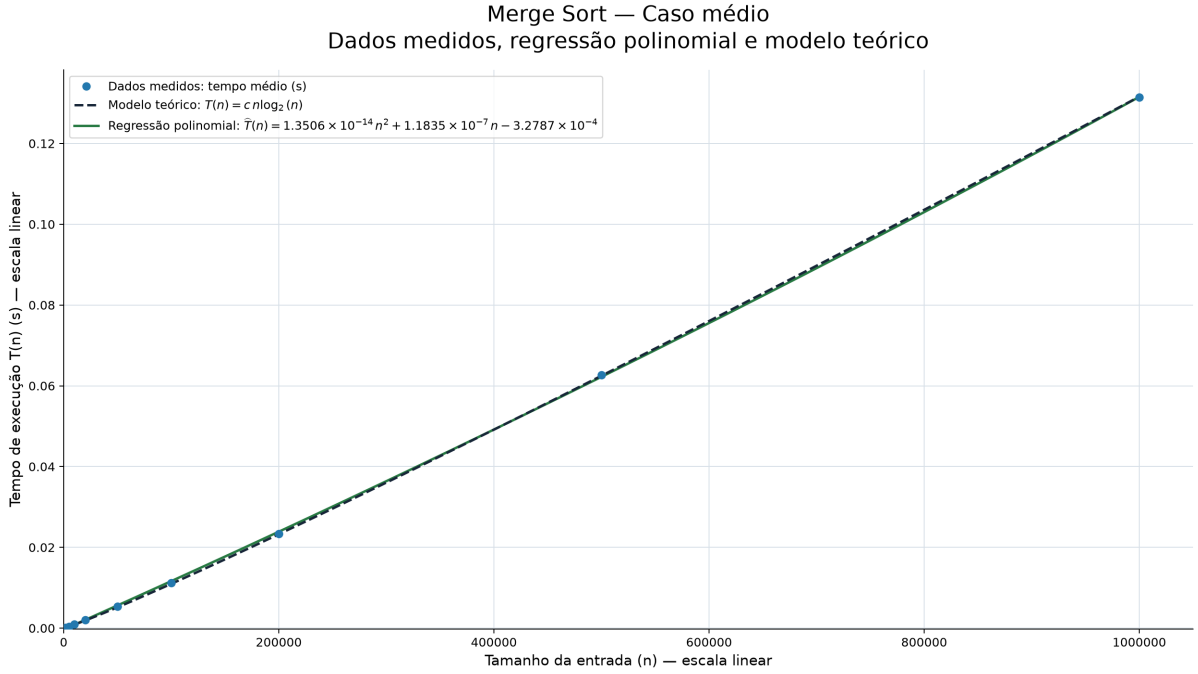


Figura 14 – Tempos de execução do Merge Sort no caso médio

Fonte: Elaboração própria.

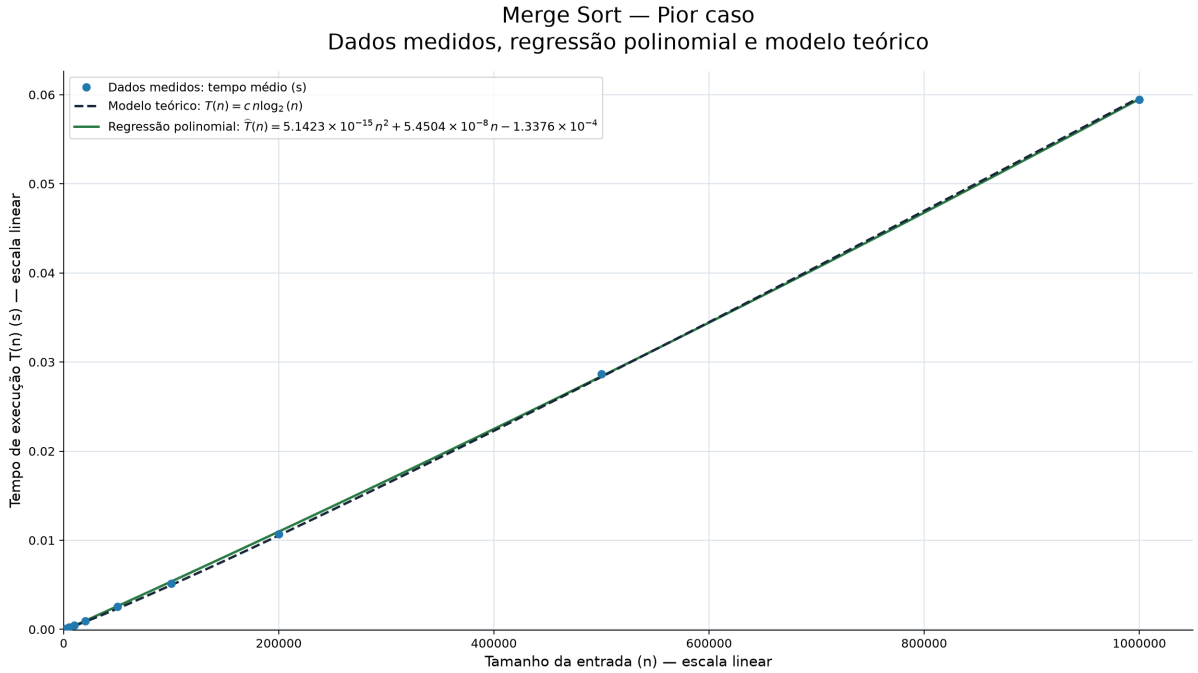


Figura 15 – Tempos de execução do Merge Sort no pior caso

Fonte: Elaboração própria.

Parte III

Análise

3 Ordem de crescimento dos algoritmos

A análise de algoritmos afirma que o algoritmo Insertion Sort possui complexidade $O(n)$ e $O(n^2)$ no melhor e no pior caso, respectivamente. Para o algoritmo Merge Sort, a complexidade $O(n \log(n))$ aplica-se a todos os casos. É possível comprovar essa afirmação ao analisar as Figuras 10, 12, 13 e 15, nas quais a curva teórica acompanha de perto a curva dos dados medidos.

Nos casos em que a curva dos dados medidos se distancia um pouco da curva teórica, esse distanciamento se justifica pela capacidade de processamento da máquina utilizada (processador, memória RAM e sistema operacional).

4 Análise ao dobrar o valor de n

Analisando os gráficos de melhor caso, caso médio e pior caso do Insertion Sort e Merge Sort, que estão disponíveis nas Figuras 1, 2, 3, 4 5 6 podemos analisar que, ao dobrar n, os tempos apresentam crescimento esperado para cada algoritmo.

No Insertion Sort, o tempo aproximadamente dobrou no melhor caso, comportamento consistente com o crescimento linear, pois $(2n)/n = 2$. Nos casos médio e pior, aproximadamente quadruplicou, conforme o crescimento quadrático:

$$\frac{(2n)^2}{n^2} = 4. \tag{4.1}$$

No Merge Sort, os tempos aumentaram um pouco mais de duas vezes nos três cenários, comportamento consistente com $n \log_2(n)$:

$$\frac{2n \log_2(2n)}{n \log_2(n)} = 2\left(1 + \frac{1}{\log_2(n)}\right). \tag{4.2}$$

Para todos os casos e cenários, tem-se a seguinte tabela:

Tabela 1 – Razão entre os tempos médios de execução para entradas de 500.000 e 1.000.000 de elementos

Algoritmo	Cenário	Razão observada
Insertion Sort	Melhor caso	2,00
Insertion Sort	Caso médio	4,00
Insertion Sort	Pior caso	4,04
Merge Sort	Melhor caso	2,09
Merge Sort	Caso médio	2,10
Merge Sort	Pior caso	2,07

Fonte: Elaboração própria.

4.1 Análise da diferença significativa dos algoritmos

As Figuras7, 8 e 9, referentes aos gráficos comparativos dos algoritmos no melhor caso, no caso médio e no pior caso, respectivamente, evidenciam que a diferença entre os tempos de execução se torna mais significativa nos casos médio e pior conforme o tamanho da entrada aumenta. Para uma entrada de 50 000 elementos, o tempo médio do Insertion Sort foi aproximadamente 114 vezes maior que o do Merge Sort no caso médio e 477 vezes maior no pior caso.

No melhor caso, o Insertion Sort apresentou tempos de execução menores que os do Merge Sort. Embora diferenças pequenas possam ser pouco perceptíveis em uma execução isolada, sua relevância depende da quantidade de execuções e dos requisitos da aplicação.

4.2 Regressão Polinomial

A regressão polinomial foi aplicada aos tempos médios de execução dos dez tamanhos de entrada, considerando as 50 repetições de cada combinação de algoritmo e cenário. Os coeficientes foram calculados pelo método dos mínimos quadrados, que minimiza a soma dos quadrados das diferenças entre os tempos medidos e os estimados.

Foi utilizado um polinômio de primeiro grau para o melhor caso do Insertion Sort e de segundo grau para os demais cenários. Nas equações apresentadas, n representa o tamanho da entrada e $\widehat{T}(n)$ representa o tempo estimado em segundos.

4.2.1 Insertion Sort

Para o melhor caso, foi obtida a função:

$$\widehat{T}(n) = 1{,}4868 \times 10^{-9} n - 2{,}3227 \times 10^{-6}. \tag{4.3}$$

Para o caso médio, foi obtida a função:

$$\widehat{T}(n) = 2{,}4521 \times 10^{-10} n^2 + 1{,}3458 \times 10^{-7} n - 3{,}6176 \times 10^{-3}. \tag{4.4}$$

Para o pior caso, foi obtida a função:

$$\widehat{T}(n) = 4{,}9654 \times 10^{-10} n^2 - 3{,}3110 \times 10^{-6} n + 9{,}7743 \times 10^{-2}. \tag{4.5}$$

A função obtida para o melhor caso apresenta crescimento linear, compatível com o comportamento teórico esperado. Nos casos médio e pior, o termo quadrático predomina para as maiores entradas, em concordância com o crescimento esperado de n^2.

As Figuras 10, 11 e 12 apresentam os dados medidos, as regressões e os modelos teóricos ajustados.

4.2.2 Merge Sort

Para o melhor caso, foi obtida a função:

$$\widehat{T}(n) = 5{,}4099 \times 10^{-15} n^2 + 5{,}3162 \times 10^{-8} n - 7{,}3106 \times 10^{-5}. \tag{4.6}$$

Para o caso médio, foi obtida a função:

$$\widehat{T}(n) = 1{,}3506 \times 10^{-14} n^2 + 1{,}1835 \times 10^{-7} n - 3{,}2787 \times 10^{-4}. \tag{4.7}$$

Para o pior caso, foi obtida a função:

$$\widehat{T}(n) = 5{,}1423 \times 10^{-15} n^2 + 5{,}4504 \times 10^{-8} n - 1{,}3376 \times 10^{-4}. \tag{4.8}$$

As funções polinomiais aproximam os dados no intervalo de entradas avaliado. Entretanto, esses ajustes não atribuem complexidade quadrática ao Merge Sort, cujo crescimento teórico é $n \log_2(n)$ nos três cenários.

As Figuras 13, 14 e 15 permitem comparar as regressões com os dados medidos e o modelo teórico. A proximidade entre esses elementos, juntamente com o aumento de pouco mais de duas vezes no tempo ao dobrar as maiores entradas, indica compatibilidade dos resultados experimentais com o comportamento linearítmico esperado.

Conclusão

A implementação dos algoritmos Insertion Sort e Merge Sort e a análise das três mil medições permitiram comparar os resultados experimentais com os modelos teóricos. O Insertion Sort apresentou crescimento linear no melhor caso e quadrático nos casos médio e pior. O Merge Sort apresentou comportamento compatível com o crescimento linearítmico nos três cenários.

Ao dobrar o tamanho das maiores entradas, o tempo do Insertion Sort aproximadamente dobrou no melhor caso e quadruplicou nos casos médio e pior. Para o Merge Sort, o tempo aumentou pouco mais de duas vezes. Na comparação entre os algoritmos, o Insertion Sort foi mais rápido para os vetores previamente ordenados, enquanto o Merge Sort apresentou vantagem nos casos médio e pior, principalmente nas maiores entradas.

As regressões polinomiais permitiram obter funções que aproximam os tempos medidos e compará-las aos modelos teóricos. No caso do Merge Sort, o ajuste polinomial descreve os dados no intervalo avaliado, sem alterar sua complexidade teórica. Dessa forma, os objetivos da atividade foram alcançados, e os resultados evidenciaram a importância de considerar o tamanho da entrada e a organização inicial dos dados na escolha do algoritmo de ordenação.